

MTK86系列平台基于虚拟化op-tee使用手册

1. 概述

该文档介绍了 OP-TEE 的功能、结构设计、各组件的组成、如何编写 CA 和 TA，以及如何自测，如何 Debug。

本文档的章节安排为：

第 1 章为概述，介绍 TrustZone 的 OP-TEE 的基本概念。

第 2 章介绍 OP-TEE 的框架和功能。

第 3 章介绍如何开发基于 OP-TEE 的应用。

1.1 TrustZone 功能简介

TrustZone 是 Arm A-profile 架构中的安全架构名称。TrustZone 提供了两个执行环境，并在它们之间通过系统范围的

硬件强制隔离，如 Error! Reference source not found.所示：

Error! Reference source not found.为普通世界和可信世界的示意图。普通世界运行一个丰富的软件栈。这个软件栈

通常包括大量的应用程序集、一个复杂的操作系统（如 Linux），以及一个可能的虚拟机监控器。这样的软件栈庞

大且复杂。尽管可以采取提高其安全性，但由于攻击面较大，它们更容易受到攻击。可信世界运行一个更小

且更简单的软件栈，称为可信执行环境（TEE）。通常，TEE 包括由轻量级内核托管的几个可信服务。可信服务提

供诸如密钥管理等功能。这个软件栈的攻击面显著较小，由于这个软件栈的攻击面显著较小，因此其脆弱性大大减小

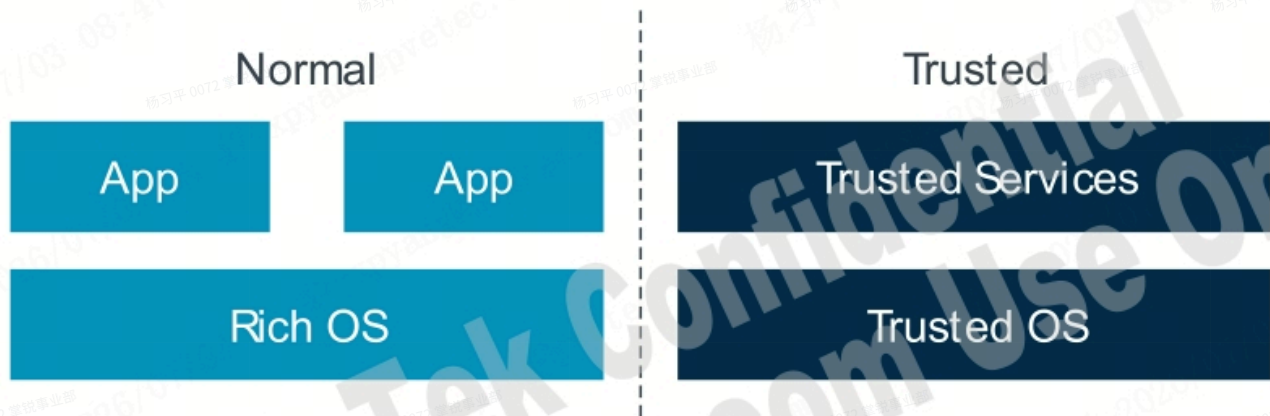


图 1-1. 普通世界和可信世界

Armv8 和 v9 将 CPU 分为 4 种异常等级，分别是 EL0，EL1，EL2 和 EL3。

在 EL0、EL1 和 EL2 级别，处理器可以处于安全状态或非安全状态，这由 SCR_EL3.NS 位控制。通常会看到如下写

法：

NS-EL1: 非安全状态，异常级别 1

S-EL1: 安全状态，异常级别 1

EL3 始终处于安全状态，无论 SCR_EL3.NS 位的值如何。

安全状态和异常级别的安排如图 1-2 所示：

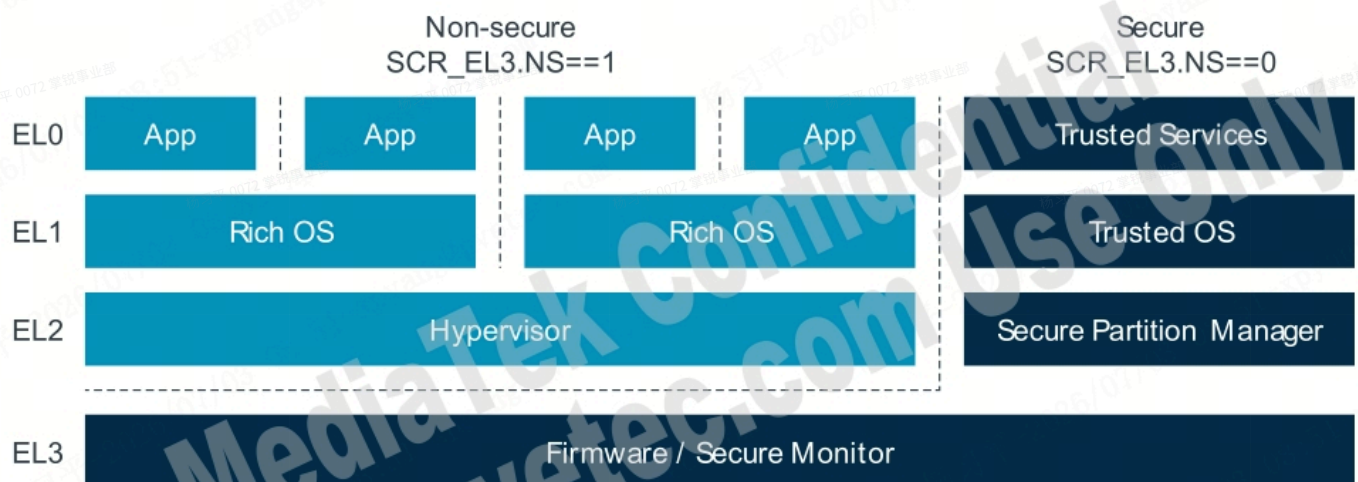


图 1-2. Arm 异常等级划分

Arm 为了保证 TrustZone 的安全性，提供了一些列的硬件隔离技术，包含中断隔离，片上 RAM/ROM 隔离，片外

RAM/ROM 隔离，外围设备的硬件隔离，外部 RAM 和 ROM 的隔离等。Arm 提供的隔离技术可以参阅 Arm 的技术文档，在此不再赘述。

MediaTek 提供了功能更加强大的内存保护单元 (SMPU) 保证 TrustZone 所在的内存区域只能被安全的 CPU 访问，阻止非安全的 CPU，以及各种外设通过各种方式访问 TrustZone 所在的内存区间。

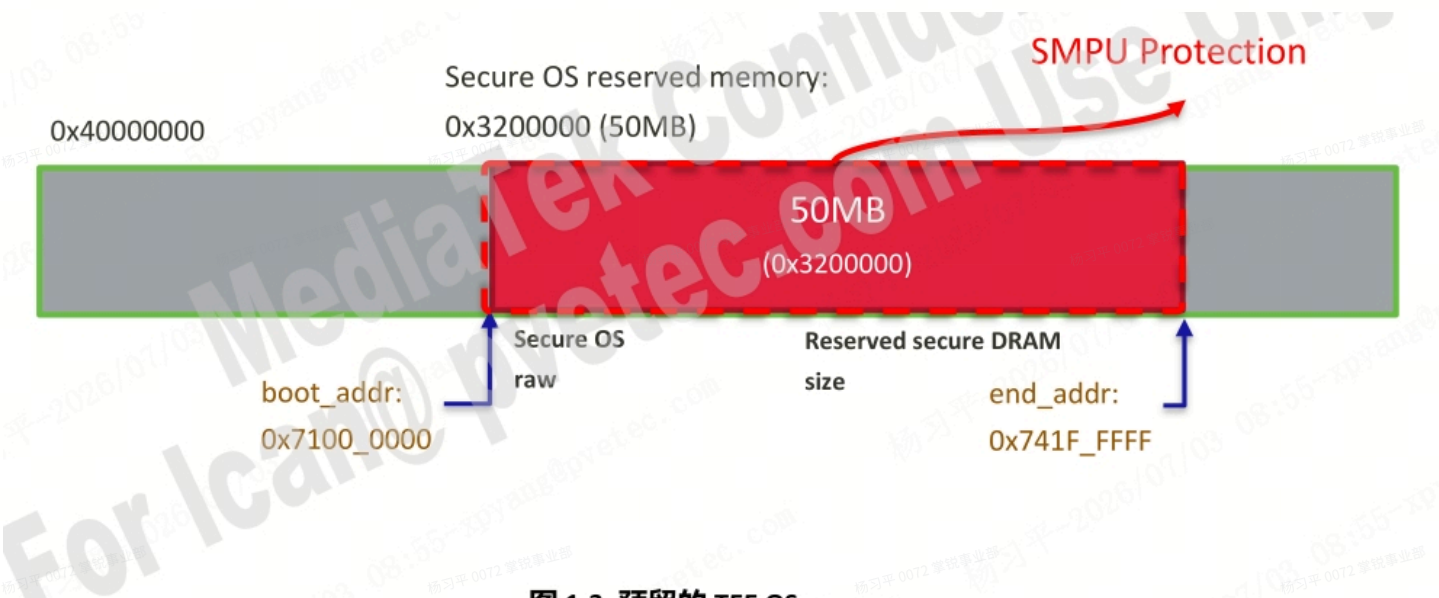


图 1-3. 预留的 TEE OS memory

图 1-3 是预留的 TEE memory 和 SMPU 保护这段 reserved memory 的示例图。这段 memory 只允许 Secure CPU 访问，即对应前文，只有 CPU 处在 S-EL0, S-EL1 或 EL3 时才可以访问;其他场景，例如 user space 程序，Linux Kernel 程序，hypervisor 程序都无权访问此段 memory。

1.2 OP-TEE 简洁

OP-TEE 是一个开源的 Secure OS。图 1-4 是 OP-TEE 官方提供的架构图。

OP-TEE 内核是运行在 S.EL1 和 S.EL0 中的可信应用程序。可信应用程序通过 TEE 内部 API 与 OP-TEE 内核通信。TEE 内部 API 是由 GlobalPlatform 组织开发的标准 API。GlobalPlatform 致力于开发标准 API，这些 API 不仅支持 OP-TEE，还支持许多不同的 TEE。采用 GP 标准意味着在其他支持 GP 接口的 TEE 应用可以方便地移植到 OP-TEE 上，反之亦然，为 OP-TEE 设计的 TEE 应用也可以方便地移植到其他支持 GP 接口的 TEE 上。

在非安全状态下，内核空间中有一个 OP-TEE 驱动程序。它负责处理与 OP-TEE 内核的低级别通信。在非安全用户空间 (EL0) 中，有一个用户空间库 (libopenteec.so) 实现了 GlobalPlatform API。TEE Client API 是应用程序用来访问 TA (可信应用程序) 的接口。

OP-TEE 还包括一个称为 tee-supplciant 的组件。tee-supplciant 处理 OP-TEE 需要返回到 REE world 处理的事务，比如从文件系统中加载 UTA，再比如 TEE 安全存储功能需要把加密后的数据保存在 Flash 或者 RPMB 中，但 TEE 本身无法读写 flash 或 RPMB，需要切回 REE，通过 tee-supplciant 完成读写。

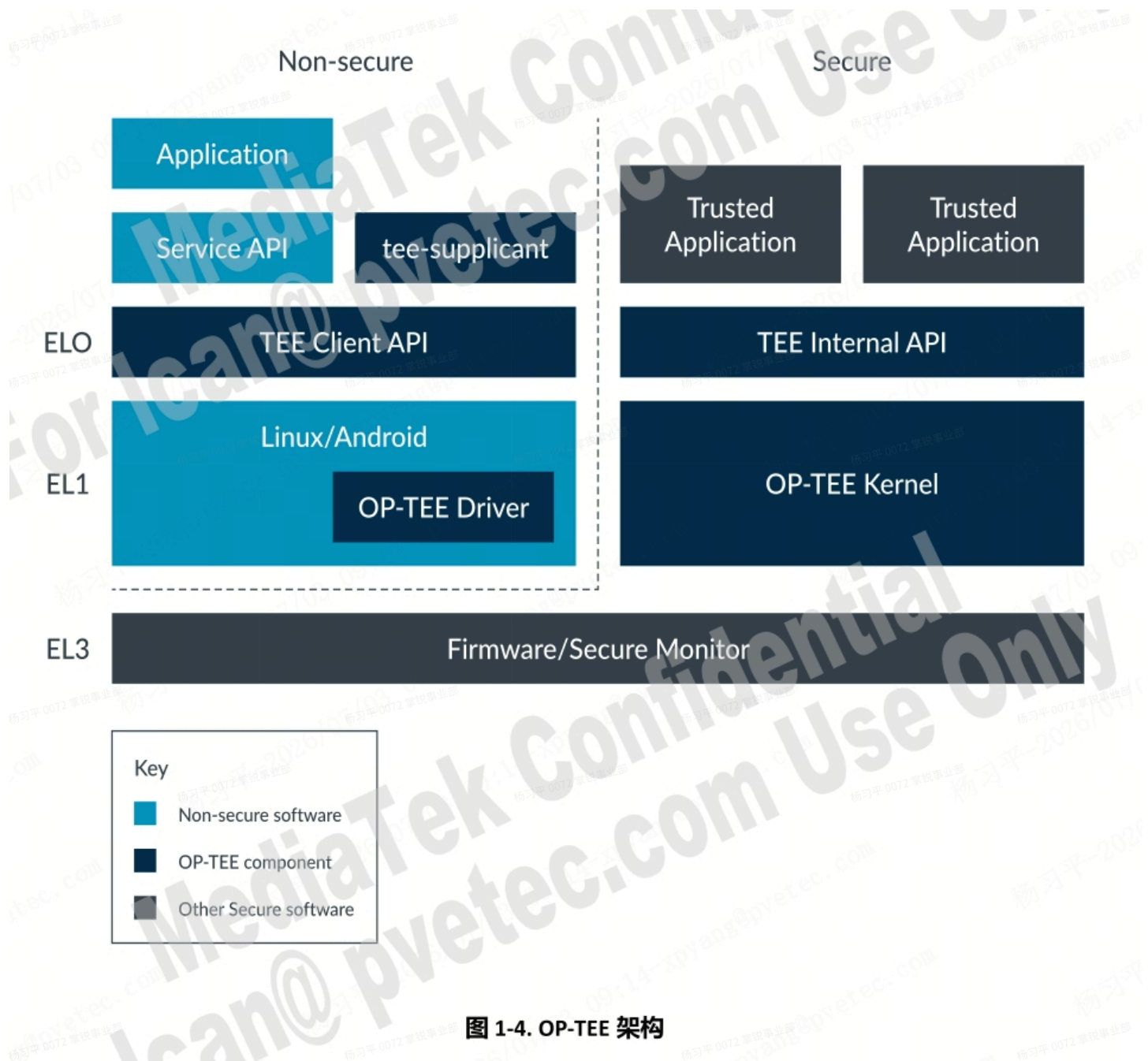


图 1-4. OP-TEE 架构

2. OP-TEE 框架和功能

2.1 开启虚拟化optee核心组件

在章节 1.2 中我们简单提到了 OP-TEE 的构成，本章主要介绍开启虚拟化后 OP-TEE 的构成。

图 2-1 是未开启虚拟化时 OP-TEE 的组成，各组件分别为：

- Tee-supplciant: TEE 守护进程处理 load UTA, secure storage 读写等需要切回 REE 执行的事务
- libopenteec.so: 提供 Client 端 GP API, CA 与 TA 通讯时使用
- OP-TEE Linux Driver: 提供 shared memory 等 API, 执行 SMC, 与 Secure World 通讯
- OP-TEE OS: OPTEE Kernel 层提供的所有功能

- PTA (Pseudo Trusted Applications)：提供 CA 或 UTA 可以调用的 API，一些系统服务会以 PTA 的方式提供，PTA 运行在 OP-TEE Kernel 层，与 OP-TEE OS 打包在一起

基于 OP-TEE 的应用：CA & TA 为运行在 OP-TEE 上的应用

- CA (Client Agent)：运行在 REE (非安全世界)
- TA (Trusted App)：运行在 TEE (安全世界)，需要受保护的操作和密钥可以放于 TA 中

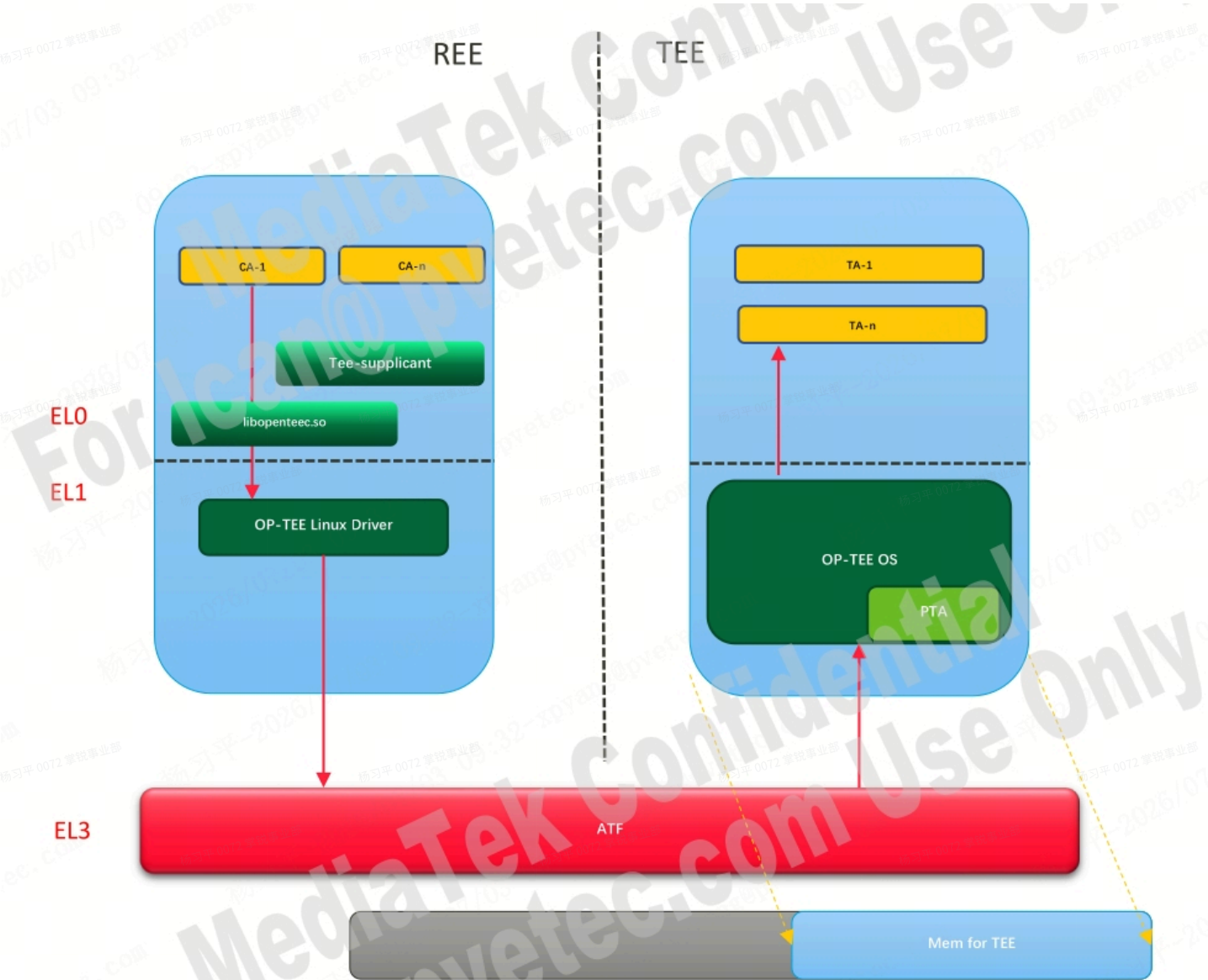


图 2-1. 非虚拟化下 OP-TEE 的组成

未开启虚拟化时，OP-TEE 的初始化在 ATF 跳转 BL32 时进行。

当开启 OP-TEE 虚拟化之后，OP-TEE 会在 TEE 内，为每一个 Host VM，创建一个对应的 OP-TEE 的 VM。

如图 2-2 所示，我们的 L+L+A 架构包含两个 Linux 的虚拟机和一个 Android 的虚拟机，分别为

- SOS (Linux 虚拟机)

- UOS (Android 虚拟机)

- UOS (Linux 虚拟机)

开启虚拟化后，ATF 跳转 BL32 做初始化时，只会初始化 OP-TEE nexus core 层。

在每个虚拟机创建之前，Hypervisor 层会发送 SMC command (OPTEE_SMC_VM_CREATED) 至 OP-TEE，创建出对应的 OP-TEE 虚拟机，并在预先分配好的安全内存中初始化 OP-TEE VM 所需的数据结构。

SOS (Linux 虚拟机) 的 TEE 请求会进入到 OP-TEE-VM1 中，OP-TEE-VM1 通过 RPC 或者 RET 返回到 REE 时，也会回

到 SOS 域

UOS (Android 虚拟机) 同理，请求会进入到 OP-TEE-VM2，从 TEE 返回时返回到 UOS (A)

UOS (Linux 虚拟机) 同理。

OP-TEE-VM1/VM2/VM3 在物理内存上是完全隔离的，互不影响。

2.2 MediaTek 平台 OP-TEE 开启方式

若需要开启或关闭 OP-TEE，需同时修改 Yocto 和 Android branch。下面分别介绍 Yocto branch 和 Android branch 需要做哪些改动。

2.2.1 Yocto branch

请先检查如下目录存在；若如下目录不存在，则表示 release 不完全，需联系 MediaTek 处理。

如下4.7.0 是68 的，其他的一般是3.18.0

- src/bsp/trustzone/optee/4.7.0/optee_client
- src/bsp/trustzone/optee/mtk_ext/client
- src/bsp/trustzone/optee/mtk_driver/secure_driver_spmlib
- src/bsp/trustzone/optee/mtk_ext/os_spmlib
- src/bsp/trustzone/optee/mtk_ext/plat_spmlib
- prebuilt/bsp/trustzone/optee/4.7.0/optee_os

确认一下目录都存在后，下面是 OP-TEE 的 config 配置方法：

Yocto branch 与 OP-TEE 相关的 Config 有两个，功能分别是：

TEE_SUPPORT：表示支援哪种 TEE，填入 “optee” 表示开启 OP-TEE。

TEE_TEST_SUPPORT：表示是否需要在编译时打包 OP-TEE 测试工具 xtest，此 config 只针对 OP-TEE 有效，默认为开启

确认一下目录都存在后，下面是 OP-TEE 的 config 配置方法：

Yocto branch 与 OP-TEE 相关的 Config 有两个，功能分别是：

TEE_SUPPORT：表示支援哪种 TEE，填入“optee”表示开启 OP-TEE。

TEE_TEST_SUPPORT：表示是否需要在编译时打包 OP-TEE 测试工具 xtest，此 config 只针对 OP-TEE 有效，默认为开启

请客户根据自己的 project，选择对应的位置修改。

若需要开启 OP-TEE，请将 config 设定为：

```
TEE_SUPPORT = "optee"
```

若需要关闭 OP-TEE，请将 config 设定为：

```
TEE_SUPPORT = "none"
```

若需要开启 OP-TEE，请将 Kernel dts 中 OP-TEE 节点的 disabled status 删除或者改为 okay status：

```
optee {  
    status = "okay";  
}
```

2.2.2 Android Branch

请先检查如下目录存在；若如下目录不存在，则表示 release 不完全，需联系 MediaTek 处理

- vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_client
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/client
- vendor/mediatek/proprietary/trustzone/optee/mtk_driver/secure_driver_spmlib
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/os_spmlib
- vendor/mediatek/proprietary/trustzone/optee/mtk_ext/plat_spmlib
- vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_os_spmlib

Android branch 与 OP-TEE 相关的 config 有两个，分别的功能是

MTK_OPTEE_SUPPORT：Vendor 层是否打开 OP-TEE，若打开，则会把 OP-TEE 的 binary 打包进 tee.img。

MGVI_MTK_OPTEE_SUPPORT：HAL 层 config，此 config 不会影响 OP-TEE 的开启或关闭，但 Android 域基于 OP-TEE

的 feature 可能会基于这个 config 判断自身的开启和关闭，故建议一并开启。

开启的位置为：

MTK_OPTEE_SUPPORT 在 ProjectConfig.mk 中

MediaTek 公版示例文件路径为：

device/mediateksample/{PROJECT_NAME}/ProjectConfig.mk

(PROJECT_NAME 需替换成您使用的 Android project，例如 auto8668p1_64_vm_uos)

MGVI_MTK_OPTEE_SUPPORT 定义在 VendorConfig.mk 中，

MediaTek 公版示例文件路径为：

device/mediatek/vendor/{PROJECT_NAME}/VendorConfig.mk

(PROJECT_NAME 需替换成您使用的 Android mgvi project，例如

mgvi_auto_64_armv82_wifi_vm_uos)

若开启 OP-TEE，则将这两个 config 值为 yes

```
MTK_OPTEE_SUPPORT = yes
```

```
MGVI_MTK_OPTEE_SUPPORT = yes
```

若需要关闭 OP-TEE，将这两个 config 置为 no，或者直接删除这两个 config 的定义皆可。

Kernel 部分所需修改与 Yocto 相同，请参考上节内容，用同样的方法打开 Android 域 Kernel 层 OP-TEE。

2.3 安全存储机制

在 OP-TEE 中，安全存储是根据 GlobalPlatform 的 TEE Core API 定义实现的。该规范要求必须能够存储通用数据和密钥材料，并保证存储数据的机密性和完整性，以及修改存储的操作的原子性（这里的原子性意味着整个操作要么

成功完成，要么不进行任何写入）。

目前在 OP-TEE 中有两种安全存储实现。

2.3.1 REE 文件系统

使用外部（非安全）文件系统来存储加密的数据。这种方式依赖于正常世界（Normal World）的文件系统，但所有

的敏感操作（如加密和解密）都在安全世界（Secure World）内完成。其实现原理请参阅图 2-3。

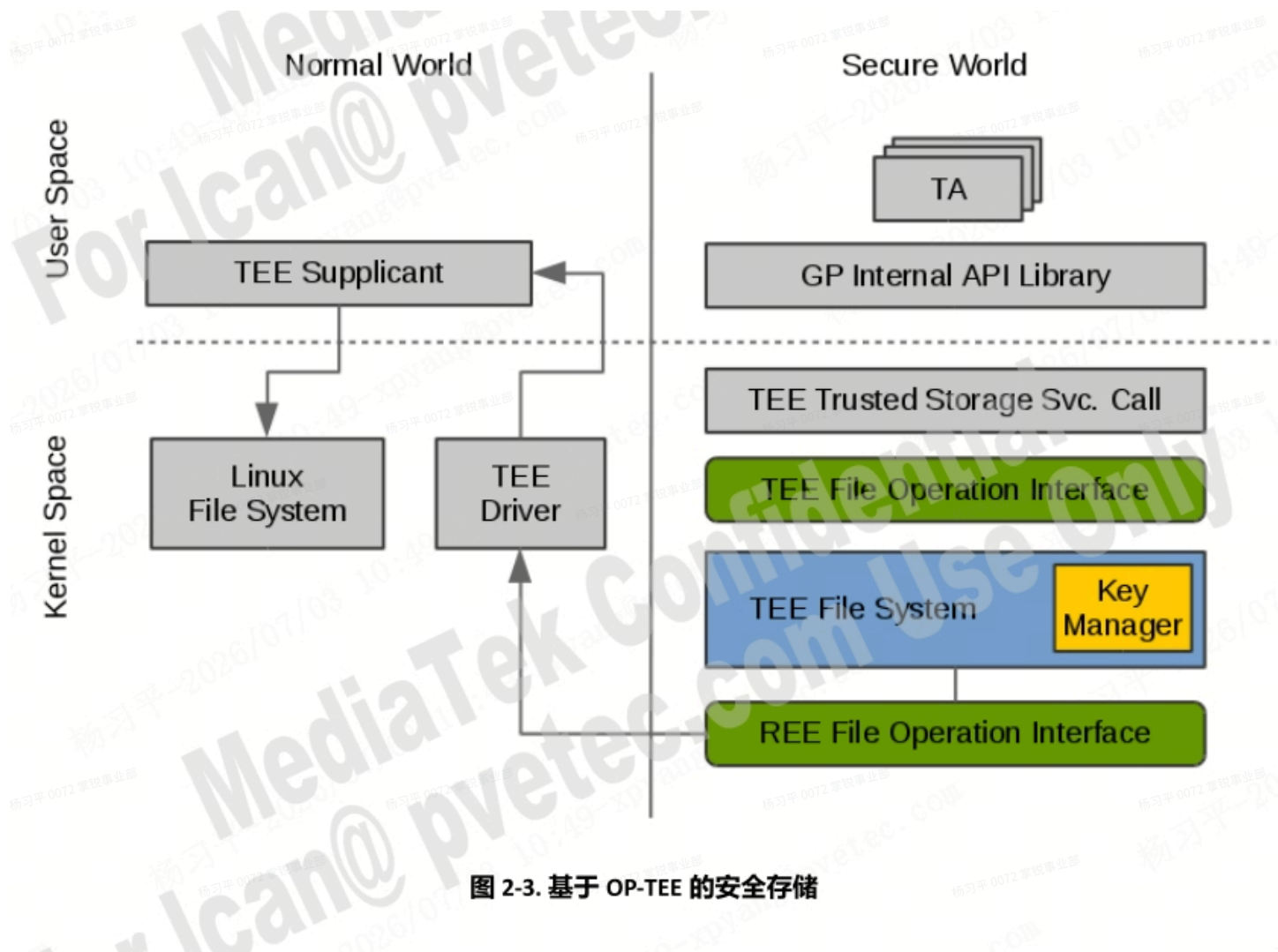


图 2-3. 基于 OP-TEE 的安全存储

OP-TEE 默认支持此种方式的安全存储。

注意：

- 使用 REE FS 形式的安全存储保存的数据，会保存在对应 VM 的文件系统中，用户或程序无法在 REE 测读取到文件的内容，
- 但有读写权限的用户或程序可能篡改或删除此文件。

Yocto 域默认存储位置为：/var/lib/tee 或/data/vendor/tee

若想修改此路径，方式为：

Yocto branch 上找到

meta/meta-mediatek/recipes-bsp/trustzone/optee-client_4.7.0.bb

修改 CFG_TEE_FS_PARENT_PATH 的定义。

Android 域默认存储路径为：/data/vendor/tee

若想修改此路径，方式为：

Android branch 上找到

vendor/mediatek/proprietary/trustzone/optee/4.7.0/optee_clnt/Android.mk

修改 CFG_TEE_FS_PARENT_PATH 的定义。

注意：

· 请读者根据需求修改此目录，修改时请留意 tee-suppllicant 是否有权限创建目录；目录所在分区是否会因重启或者恢复出厂设置而被重置。

2.3.2 RPMB 文件系统

基于 RPMB 的安全存储，数据加解密方式与 REE FS 方式类似，只是存储介质由普通文件系统变为存储在 UFS（或

eMMC）附带的 RPMB 区域，利用 RPMB 的防篡改和防重放特性，提供更高的安全性。存储原理与图 2-3 类似，只

是将左下角的“Linux File System”替换成 RPMB 的 driver。

若想在虚拟化的项目上支持基于 RPMB 的安全存储，需要 UFS 硬件支持多个 RPMB region（例如需要 UFS3.0 及以上版本，eMMC 不支持），因此此种存储方式并不是默认支持的，需联系 MediaTek 确认此方式在您的硬件上是否支持。

在编写 TA 程序时，需通过参数指定，将数据存储与 REE 文件系统和 RPMB 文件系统实现。

定义了两个 OP-TEE 特定的存储标识符：TEE_STORAGE_PRIVATE_REE 和 TEE_STORAGE_PRIVATE_RPMB。

TEE_STORAGE_PRIVATE_REE 表示将数据保存于 REE 文件系统；

TEE_STORAGE_PRIVATE_RPMB 表示将数据保存于 RPMB 文件系统。

注意：

· 无论是 REE FS 还是 RPMB 存储方式，各个 VM 存储的数据不互通；即由哪个域写入，就只能从哪个域读取。

在平台上执行 xtest 6001 测项，可以判断中平台上安全存储功能是否正常。

```
* regression_6001 Test TEE_CreatePersistentObject
o regression_6001.1 Storage id: 00000001
  regression_6001.1 OK
o regression_6001.2 Storage id: 80000000 -> REE FS 方式的安全存储
  regression_6001.2 OK
o regression_6001.3 Storage id: 80000100 -> RPMB 方式的安全存储
  regression_6001.3 OK
regression_6001 OK
```

若执行 xtest 6001 测试时，没有执行 regression_6001.3 测项，则说明当前设备并不支持 RPMB，不支持的原因可能

是 RPMB key 未烧录，或软硬件环境不满足需求；若确定有 RPMB 需求，请联系 MediaTek 做进一步排查。

有关 xtest 如何执行，可以参阅本文章节 4.1。

2.4 CA 和 TA 介绍

在 OP-TEE 中，CA 通常指的是"Client Application"（客户端应用程序）。CA 是运行在普通世界（REE，Rich Execution Environment）中的应用程序，它与运行在受信世界（TEE，Trusted Execution Environment）中的受信应用程序

（TA，Trusted Application）进行通信。CA 通过调用 TEE API 来请求 TA 执行安全操作。

以下是 CA 和 TA 之间的基本交互流程：

1. CA 启动：客户端应用程序在普通世界中启动。
2. 初始化 TEE 会话：CA 通过 TEE API 初始化与 TA 的会话。
3. 发送命令：CA 向 TA 发送命令请求，要求 TA 执行特定的安全操作。
4. TA 处理请求：TA 在受信世界中处理请求，并返回结果。
5. 接收结果：CA 接收 TA 返回的结果，并根据需要进行处理。
6. 关闭会话：操作完成后，CA 关闭与 TA 的会话。

用户可根据需求，在 user space 或 Kernel space 编写 CA。

在 OP-TEE 中，TA 指“Trusted Application”（可信应用）。OP-TEE 支持三种类型的可信应用（TA），分别是用户模式

可信应用（UTA）、静态可信应用（PTA，也称为 Pseudo TA）和早期用户模式可信应用（Early UTA）。这些应用类型在功能、执行时机和运行环境方面有所不同。下面是它们的主要区别：

1. 用户模式可信应用（UTA）

- 运行环境：UTA 在用户模式下运行，与操作系统的内核模式相对立。这意味着它们在一个隔离的环境中执行，

提供了更高的安全性。

- 部署方式：UTA 通常作为单独的应用程序部署，可以通过安全存储进行加载和管理。
- 用途：UTA 适用于处理高度敏感的数据或执行安全关键的任务，如加密操作、密钥管理等。

2. 静态可信应用（PTA）

运行环境：PTA 在 TEE 内核模式下运行，与 UTA 相比，它们有更高的权限和更直接的访问 TEE 核心服务的能

力。

- 部署方式：PTA 通常是在 TEE 操作系统构建时静态集成的，不是作为独立应用程序加载的。
- 用途：PTA 用于实现一些需要更高权限访问 TEE 资源的功能，如管理和控制 TEE 内部的其他组件或服务。

3. 早期用户模式可信应用（Early UTA）

- 运行环境：Early UTA 也在用户模式下运行，但它们在系统启动的早期阶段被加载和执行。
- 部署方式：与普通的 UTA 相似，但它们在 TEE 初始化过程中较早启动，用于处理启动早期阶段的安全任务。
- 用途：Early UTA 通常用于执行系统启动过程中的安全检查或配置，如验证启动配置的完整性或早期硬件资源的

安全配置。

UTA 会放置在文件系统中，

Yocto 域的路径是/lib/optee_armtz/。

Android 域的路径是/vendor/lib/optee_armtz/。

若一个TA 在Yocto 域和Android 域都需要，您需要在每个域的文件系统上都放置一份。

3. 基于OP-TEE 的应用开发

在 Yocto 域和 Android 域都支持 OP-TEE，且 Yocto 域和 Android 域所需的 API 都是一样的。

这意味着 Yocto 域和 Android 域的 CA/TA 代码可以完全一样。

如果您的程序需要运行在 Yocto 域，需要将 CA 和 TA 的执行文件打包进 Yocto 域的文件系统，

Android 域同理，若需要运行于 Android 域，也需要将执行文件打包进 Android 域的文件系统。

编写CA 和 TA 所依赖的GP API

OP-TEE 支持 GP (GlobalPlatform) API，意味着您的 CA/TA 程序可以方便的在其他支持 GP API 的 TEE OS 上进行快速移

植。关于 GP API 的资料，您可以自行查阅互联网上的资料。

CA 侧使用的 GP API 以 TEEC 开头，您可以在如下头文件中找到相关定义

`${PATH}/optee/4.7.0/optee_client/libteec/include/tee_client_api_extensions.h`

其中，Yocto branch 上的路径为：`src/bsp/trustzone`

Android branch 上的路径为：`vendor/mediatek/proprietary/trustzone`

CA 端负责与 TEE 通讯的 API 主要是以下五个 API：

1. `TEEC_Result TEEC_InitializeContext(`
`const char* name,`
`TEEC_Context* context)`
2. `void TEEC_FinalizeContext(`
`TEEC_Context* context)`
3. `TEEC_Result TEEC_OpenSession (`
`TEEC_Context* context,`
`TEEC_Session* session,`
`const TEEC_UUID* destination,`
`uint32_t connectionMethod,`
`const void* connectionData,`
`TEEC_Operation* operation,`
`uint32_t* returnOrigin)`
4. `void TEEC_CloseSession (`
`TEEC_Session* session)`
5. `TEEC_Result TEEC_InvokeCommand(`
`TEEC_Session* session,`
`uint32_t commandID,`
`TEEC_Operation* operation,`
`uint32_t* returnOrigin)`

CA 端与共享内存操作有关的 API 是如下三个：

1. `TEEC_Result TEEC_RegisterSharedMemory(TEEC_Context *context,`
`TEEC_SharedMemory *sharedMem);`
2. `TEEC_Result TEEC_AllocateSharedMemory(TEEC_Context *context,`
`TEEC_SharedMemory *sharedMem);`
3. `void TEEC_ReleaseSharedMemory(TEEC_SharedMemory *sharedMemory);`

GP 在 TEE 内的 API 以 “TEE_ “ 开头，涉及到 TEE 内的各种操作，例如加解密操作，安全文件读写等，请参阅 GP

相关文档，这里不再赘述。

编写一个 TA，必须实现以下五个 API。

1. TEE_Result TA_CreateEntryPoint(void)
2. void TA_DestroyEntryPoint(void)
3. _Result TA_OpenSessionEntryPoint(uint32_t param_types,
TEE_Param __maybe_unused params[4], void __maybe_unused **sess_ctx)
4. void TA_CloseSessionEntryPoint(void __maybe_unused *sess_ctx)
5. TEE_Result TA_InvokeCommandEntryPoint(void __maybe_unused *sess_ctx,
uint32_t cmd_id, uint32_t param_types, TEE_Param params[4])